Roteiro de retomada — Catálogo do karaokê

Versão 3 · 10/11/2026 (substitui a versão 2, de 27/10/2026; tarefas e aceites não mudaram).
Programação III · Ensino Médio Integrado — Técnico em Informática · IFSul — Campus Passo Fundo
Professor: Élder F. F. Bernardi · **Modalidade:** duplas **Tutor da atividade:** Prof. Leonardo Constantin

Organização: 14 duplas por turma, 150min em aula e 1h extraclasse por encontro. Quatro encontros: 600 + até 240min = 14h de desenvolvimento; apresentação em outro encontro de 150min. Setup essencial acompanhado em aula.

Página do trabalho · Enunciado

Sumário

1. Como usar este roteiro
2. Para que serve esta base
3. 1. Preparar e executar
4. 2. Encontrar cada responsabilidade
5. 3. Conferir o contrato antes de programar
6. 4. Integrar cadastro — R2 e R5
7. 5. Integrar edição — R3 e R5
8. 6. Integrar exclusão — R4 e R5
9. 7. Conferir o núcleo e publicar — R1, R6, R7 e R8
10. 8. Consolidar entre encontros — proposta de até 60min

Como usar este roteiro

Este roteiro é para a dupla que usa a **base de retomada**. Quem usa projeto próprio pode consultar as seções 3, 7 e 8.

1. Sigam as seções 1 a 6 na ordem.
2. A publicação, que está na seção 7, começa no primeiro encontro. Não esperem terminar a seção 6 para iniciar a publicação.
3. As seções 4, 5 e 6 terminam com um **Aceite**. O aceite diz o que conferir quando a tarefa está pronta. Confiram o aceite antes de seguir.
4. A seção 8 é para o trabalho extraclasse.
5. Se um passo falhar, registrem a mensagem de erro e peçam apoio em aula.

Seção	O que vocês fazem
1	Instalam a base, preparam o banco e executam.
2	Leem os arquivos e descobrem a função de cada um.
3	Conferem o formato das requisições e das respostas da API.
4	Integram o cadastro.
5	Integram a edição.
6	Integram a exclusão.
7	Conferem o núcleo e publicam.

Seção	O que vocês fazem
8	Revisam, testam e documentam entre os encontros.

↑ Voltar ao sumário

Para que serve esta base

A base de retomada é opcional e não reduz a nota. Ela está no Classroom da disciplina e em base_retomada.zip, na página do trabalho.

- A base permite retomar a integração sem reconstruir toda a infraestrutura.
- Já vêm preparados: leitura, listagem, player, API e DAOs.
- A base traz um `AGENTS.md`, que orienta o agente de IA a agir como tutor.

As tarefas da dupla são:

- integrar **cadastro, edição e exclusão na interface**;
- tratar respostas;
- atualizar a lista.

O núcleo e a compreensão exigidos são os mesmos para quem usa projeto próprio.

↑ Voltar ao sumário

1. Preparar e executar

1. Abram a pasta `base_retomada` no editor e confiram o README.
2. Usem **Node.js 22 ou superior**, compatível com as dependências fixadas na base.
3. Executem `npm ci` para instalar as dependências declaradas.
4. A partir de `.env.example`, configurem no servidor `SUPABASE_URL` e `SUPABASE_SECRET_KEY`, com orientação do professor. No computador de vocês, esses valores ficam no arquivo `.env`. O Guia do dia 1, no passo 8, mostra como criar esse arquivo.
5. Preparem um banco de demonstração separado. Sigam o quadro "Preparar o banco", logo abaixo.
6. Com ambiente e banco preparados, executem `npm start`.
7. Abram a URL indicada pelo servidor.
8. Confirmem lista, seleção e reprodução da música de exemplo.

Preparar o banco

Façam nesta ordem: primeiro as tabelas, depois a amostra. Os arquivos são rodados no SQL Editor do Supabase.

Primeiro, as tabelas. Escolham o caso de vocês:

Caso A — as tabelas da atividade anterior já existem. Não as recriem.

1. Rodem `sql/diagnostico.sql`.
2. Se alguma linha vier `ajustar`, rodem `sql/ajustes.sql`. Ele não apaga dados.
3. Depois do `sql/ajustes.sql`, rodem o diagnóstico de novo.
4. Sigam quando não houver nenhuma linha `ajustar` ou `criar`. Linhas `conferir` de coluna que aceita vazio não impedem.

5. Se aparecer uma linha `criar`, chamem o professor.

Caso B — as tabelas ainda não existem.

1. Rodem `sql/schema.sql`.

Só depois, a amostra.

1. Incluam a amostra com `sql/seed.sql`, conforme o README.

O seed desta base é aditivo, não apaga o catálogo. Não o substituam por scripts antigos de restauração que removem dados.

O passo a passo está na etapa 4 do Guia passo a passo do Supabase.

Cuidados

- Os valores reais das variáveis são secretos. Não os enviem ao Git, ao navegador, a capturas ou a mensagens.
- A interface usa a API na mesma origem, como `/api/musicas`. Por isso, ela não precisa de CORS nem de chave de banco no JavaScript público.
- Se a leitura falhar, registrem a mensagem e o ponto do fluxo antes de continuar.

↑ Voltar ao sumário

2. Encontrar cada responsabilidade

Arquivo	O que localizar durante a leitura
`app.js`	Rotas HTTP, leitura do corpo JSON, chamada de DAO e resposta.
`supabaseClient.js`	Configuração do cliente no servidor, sem copiar os valores secretos.
`karaoke/artista.js`, `musica.js`, `parte.js`	Objetos usados para representar os dados.
`karaoke/DAO/ArtistaDAO.js`, `MusicaDAO.js`, `ParteDAO.js`	Consultas, alterações e transformação de linhas do banco em objetos.
`sql/schema.sql`, `sql/seed.sql`	Relações entre tabelas e repertório inicial de demonstração.
`sql/diagnostico.sql`, `sql/ajustes.sql`	Conferir e corrigir, sem apagar dados, tabelas que já existiam.
`AGENTS.md`	Orientações para o agente de IA agir como tutor: explica e pergunta, sem entregar a solução.
`public/index.html`, `estilo.css`	Campos, lista, mensagens e botões da interface.
`public/catalogo.js`	Leitura, atualização da lista, seleção e player já preparados.
`public/admin.js`	Pontos de integração a completar: cadastro, edição, exclusão e feedback.

Para ler o código em dupla:

1. Uma pessoa acompanha o clique até a rota.
2. A outra acompanha a rota até o DAO e o banco.
3. Depois, troquem.

4. Expliquem o caminho de volta à tela.

Se vocês vieram do repositório da aula

A base tem os mesmos arquivos do repositório da aula, mas alguns ficam em outras pastas. O código dos DAOs é o mesmo; só mudam os caminhos dos `require`.

Arquivo	Repositório da aula	Base de retomada
`supabaseClient.js`, `app.js`	raiz	raiz
`Artista`, `Musica`, `Parte`	`karaoke/artista.js`, `karaoke/musica.js`, `karaoke/parte.js`	`karaoke/artista.js`, `karaoke/musica.js`, `karaoke/parte.js`
`ArtistaDAO`, `ParteDAO`, `MusicaDAO`	`DAO/`	`karaoke/DAO/`
testes	`testes/`	`testes/` (a base já traz os seus)
SQL	`sql/`	`sql/`

Exemplo: o `MusicaDAO.js` importa o cliente com `require('../supabaseClient')` no repositório da aula e com `require('../../supabaseClient')` na base.

O Guia passo a passo do Supabase traz no Anexo a tabela completa de `require` e como calcular o caminho em projeto próprio.

↑ Voltar ao sumário

3. Conferir o contrato antes de programar

Contrato é o formato combinado das requisições e das respostas da API.

Operação	Contrato da base
Listar	`GET /api/musicas`: itens com `id`, `nome`, `artista` e `totalPartes`.
Consultar	`GET /api/musicas/:id`: música e suas partes; cada parte tem `letra`, `tempoEspera`, `tag` e `ordem`.
Cadastrar	`POST /api/musicas`: JSON `{ "nome": "Música de teste", "artista": "Artista de teste" }`.
Editar	`PUT /api/musicas/:id`: os mesmos campos de texto do cadastro.
Excluir	`DELETE /api/musicas/:id`: ID da música escolhida no caminho da URL.

Atenção:

- No envio, `artista` é uma **string**.
- Na resposta de música, `artista` é um **objeto** com `id` e `nome`.
- Para preencher um campo de texto, localizem o nome dentro do objeto.
- Não enviem o objeto inteiro no POST/PUT.

- Leiam a rota e a resposta real para conferir status e mensagem de erro.

↑ Voltar ao sumário

4. Integrar cadastro — R2 e R5

1. Em `admin.js`, localizem a tarefa do formulário e os campos existentes no HTML.
2. Leiam nome/artista.
3. Verifiquem os obrigatórios.
4. Impeçam recarregamento involuntário do formulário.
5. Planejem um POST com corpo JSON e o cabeçalho correspondente.
6. Aguardem a resposta.
7. Separem resposta de sucesso de erro HTTP/falha de conexão.
8. Só anunciem cadastro após confirmação.
9. Reutilizem a atualização da lista já fornecida.
10. Anotem ID e resultado de uma nova consulta.

> **Pergunte ao agente.** *"Explique o que acontece se eu esquecer o evento.preventDefault() no submit do formulário. Antes de responder, peça que eu preveja."* Depois: *"Qual a diferença entre a resposta ter response.ok falso e o fetch lançar uma exceção? Dê um exemplo de cada situação no meu projeto. Use como fonte a seção 4 do roteiro_retomada.md e os Padrões 3 e 4 do guia_interface_crud.md, na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/."*

Aceite:

- O registro aparece após recarregar a página.
- O registro permanece após reiniciar o servidor.
- Nome ou artista vazio não gera cadastro válido.

Depois do aceite, troquem a condução.

↑ Voltar ao sumário

5. Integrar edição — R3 e R5

1. Localizem como guardar o ID selecionado e preencher nome e `artista.nome`.
2. Diferenciem o estado de edição do cadastro.
3. Permitam cancelar a edição.
4. Enviem PUT ao ID selecionado.
5. Confiram a resposta.
6. Atualizem a lista.

> **Pergunte ao agente.** *"Estou no modo edição e, ao salvar, é criada uma música nova em vez de alterar a existente. Que perguntas devo me fazer para achar o erro? Use como fonte a seção 5 do roteiro_retomada.md e o Padrão 5 do guia_interface_crud.md, na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/."*

Aceite:

- O mesmo ID passa a ter o novo nome/artista após nova consulta.
- Editar não cria outra música.

Expliquem onde ID e campos entram na requisição.

↑ Voltar ao sumário

6. Integrar exclusão — R4 e R5

1. Identifiquem o registro.
2. Peçam confirmação simples antes do DELETE.
3. Aguardem resposta.
4. Mostrem erro quando houver.
5. Atualizem lista/seleção após sucesso.
6. Testem cancelamento e exclusão usando uma música criada para teste.

> **Pergunte ao agente.** *"Revise a minha função de exclusão (colada abaixo). Aponte a linha do problema e explique, sem reescrever a função. Use como fonte a seção 6 do roteiro_retomada.md e o Padrão 6 do guia_interface_crud.md, na página do trabalho: https://elderbernardiifsul.github.io/programacao-iii/integrador-2026/."*

Aceite:

- Cancelar preserva o dado.
- Confirmar remove o registro em nova consulta.

Preservem a música do seed usada para demonstrar o player.

↑ Voltar ao sumário

7. Conferir o núcleo e publicar — R1, R6, R7 e R8

- Cadastrem música sem partes: ela é válida, pode ser editada/excluída, mostra a situação e mantém Tocar indisponível.
- Reproduzam uma música com partes fornecidas; confiram vínculo e ordem, sem adicionar letras como requisito.
- Iniciem a configuração da publicação no primeiro encontro e retomem o CRUD na URL pública conforme os marcos.
- Registrem ID antes/depois de reiniciar o serviço; expliquem por que uma lista local não comprova persistência.

- Completem README com execução, URL, variáveis sem valores e uma decisão; registrem um teste por marco.

Materiais para esta seção:

- Interface com `fetch`: Guia de interface — CRUD na tela com fetch.
- Publicação: Guia de deploy na Render, que traz o passo a passo.
- Todos os guias e arquivos para baixar: página do trabalho e Classroom da disciplina.

Verificação individual:

- Cada pessoa acompanha uma operação e realiza ou prevê uma pequena alteração equivalente, com consulta.
- Vocês podem explicar oralmente, apontando o código ou por anotação ou desenho acompanhado de demonstração.

Se algo não funcionar: registrem o primeiro ponto que falhou e peçam apoio em aula. O trabalho essencial tem espaço em aula.

↑ Voltar ao sumário

8. Consolidar entre encontros — proposta de até 60min

Após o marco	Tarefa delimitada de revisão/teste/documentação
M1	Rever mapa do fluxo (20min), completar execução no README (20min), anotar dúvida/próximo passo (20min).
M2	Repetir cadastro/reconsulta (20min), registrar teste (20min), revisar trecho para explicar (20min).
M3	Repetir checklist conhecido (30min), registrar primeira falha (15min), preparar explicação (15min).
M4	Revisar README (20min), ensaiar percurso (20min), organizar evidências (20min).

- Se houver impedimento de acesso, registrem para organizar apoio com o professor.
- Não ampliem a tarefa além da hora prevista.
- Não é necessário adicionar extras.

↑ Voltar ao sumário